

Speech-Recognition

Midterm 03 Chatbot with LLM Lab, Part of LN / midterm, see iLearn course!

Prof. Dr.-Ing. Udo Garmann

Abstract

Experiments about a Chatbot with a local LLM from Hugging Face

1 Introduction

We want to create a chatbot that is able to answer questions about the THD/DIT, especially about the International Office or the Faculty of Computer Science. The questioner is a student and/or a person who is interested in studying at DIT.

If you have participated in the course “Assistant systems” and started with a chatbot there, you may take it as a starting point. Otherwise use `code_02_chatbot.py`, remove the speech images generations and extend the code so that the following requests are fulfilled.

Important: Depending on the models you choose, the starting and/or the interactions of the chatbot may take time. So it is important to plan your steps first (which approach, model, examples, ...), then implement your approach and finally answer the questions of this midterm.

2 Requests

- First plan the dialogs with the questioner and the chatbot. Write down at least 5 example dialogs with at least 3 user/chatbot interactions each in a file named ‘example-dialogs.txt’.
- Create a streamlit app with a chatbot on basis of `code_02_chatbot.py`.
- Your chatbot must use a local Hugging Face LLM. Find an appropriate model for your local hardware. Further descriptions and examples see description in the next chapter.
- A logging function has to be implemented so that the conversations are saved into a file. The name of the logging file must be `chatbot_history.log`.
- You have the following options for your experiments:
 - Intent recognition compared with open dialog
 - Open dialog without and with Retrieval-Augmented Generation (RAG)
- If you choose the intent recognition, choose one model for intent recognition and one for the open dialog. As a fallback you may use an open answer for an unknown intent.
 - start with open dialog (‘text-generation’ pipeline)
 - compare with example dialogs
 - intent classification (‘text-classification’ pipeline)
 - intent classification with fallback
- If you choose open dialog
 - start with normal model, compare dialogs
 - extend with RAG, compare dialogs
- Find out and describe some ideas or suggestions for improving the results.

Create a ZIP file with the following content and submit it via iLearn:

- example-dialogs.txt
- app.py for the streamlit app
- chatbot_history.log

- A README.md file with an analysis of the development process
- A file requirements .txt that lists the modules you have used and can be used with `pip install -r.`

Important: The LLM model must not be included in the ZIP archive.

The README.md file must have the following structure:

```
lastname, firstname, matno
```

```
Chatbot short description
```

```
# Chatbot long description
```

```
A more detailed description of the task and the approach to solve it.
```

```
# Used Hardware description
```

```
Descriptions of:
```

- CPU
- RAM
- Graphic card if any

```
# Discussion of the results
```

```
Take some dialogs from 'chatbot_history.log' and describe whether the chatbot has solved
the objective. If there are positive and negative examples of your chatbot, take at
least one each and discuss them.
```

```
# Description of technical problems - if any - during the development process
```

```
# Suggestions for improvements
```

3 Recommded models

The model you use must be appropriate for your hardware and the task. The preferred language must be English or German. Here is a list of some models, which you may test:

- `model_id = "distilbert/distilgpt2"`
- `model_id = "HuggingFaceTB/SmolLM-135M-Instruct"`
- `model_id = "google/flan-t5-small"`

More models can be used on the Hugging Face Website. Go to the models page, reduce the number of parameters and select the task (one of 'text-classification' or 'text-generation').